INTERNATIONAL
UNIVERSITY OF
APPLIED SCIENCES

WRITTEN ASSIGNMENT

University of Applied Science - Online

Study-branch: MSc. Artificial Intelligence (120 ECTS)

Advanced Statistics Workbook (DLMDSAS01)

Himanshu Saxena

Matriculation Number: 9219463

Tutor: Professor Dr. Paul Libbrecht

Realized with input of the Parameter Generator: 6bc547274ca79f20d522a744aafa677fcbf7002b

Submission date: 27.11.2025

I Abstract

This workbook documents the comprehensive application of core concepts in *Advanced Statistics (DLMDSAS01)*, covering probability distributions, parameter estimation, hypothesis testing, and Bayesian statistics, utilizing a personalized dataset specified by the ξ parameters.
The initial tasks involved fundamental analysis of probability distributions and descriptive visualization:

- **Task 1** analyzed basic probabilities and visualizations. Since $\xi_1 = 3$, the task involved modeling the number of meteorites falling into an ocean using a *geometric distribution* counting the number of Bernoulli trials until success, with $p = \xi_2 = 0.61$. This task required calculating the probability mass function (PMF) up to the point where the probability drops below 0.5%, calculating the *expectation* and *median*, and displaying these results graphically.

- **Task 2** derived the *Probability Density Function (PDF)*, computed the waiting time probability, and analyzed the location and dispersion parameters (mean, variance, and quartiles) for a continuous waiting time variable (Y) modeled by a mixture of exponential functions. This task included visualizing the PDF and a discrete histogram.

- **Task 3** applied the theory of *Transformed Random Variables* to derive the Gamma distribution of the total failure bandwidth ($T = S_1 + S_2$) for a dual-router system, based on the assumption of independent and identically distributed (i.i.d.) router failures. The model parameter (θ) was estimated using *Maximum Likelihood Estimation (MLE)* by maximizing the derived log-likelihood function.

The subsequent tasks demonstrated advanced inferential and predictive modeling techniques:

- **Task 4** performed a *Hypothesis Test* to assess whether a new factory system produced significantly higher hammer weights ($\mu_{new} > 904$ g), as mandated by $\xi_{13} = 2$. This involved proposing null (H_0) and alternative (H_1) hypotheses, constructing the Z-score test statistic, defining a decision rule based on critical values, and calculating error probabilities.

- **Task 5** tackled model complexity and overfitting through *Regularized Regression*, fitting a *10th-degree polynomial* (as $\xi_{15} = 2$) to highly variable sample data (ξ_{16}). Solutions derived using Ordinary Least Squares (OLS) were explicitly contrasted with the shrinkage effect achieved via *Ridge regularization* (L2 penalty), emphasizing the trade-off between bias and variance.

- **Task 6** focused on *Bayesian Estimation*, determining the posterior distribution of the scale parameter (θ) for a Gamma distributed variable. Utilizing the property of *conjugate priors* (Inverse Gamma family), the posterior distribution was derived and used to calculate Bayes point estimates associated with both the square-error loss function (mean) and the mode of the posterior distribution.

The methodology throughout employed rigorous mathematical derivation and utilized Python tools for computation, estimation, and comprehensive *data visualization*. This report successfully demonstrated proficiency in applying and interpreting both frequentist and Bayesian approaches to statistical inference.

Contents

II List of Figures

III List of Tables

IV Abbreviations

AFL	American Fuzzy Lop
API	Application Programming Interface
BIOS	Basic Input/Output System
Brick	Binary Run-time Integer Based Vulnerability Checker
CaaS	Container as a Service
CAB	Change Advisory Board
CE	Community Edition
CI	Continuous Integration
CLI	Command Line Interface
CNCF	Cloud Native Computing Foundation
CRED	C Range Error Detector
Dev	Development, the development team

1 Latex

1.1 Tools

MiKTeX: https://miktex.org/download TeXLive: http://tug.org/texlive/ (or alternative LaTeX-systems).

A good editor is essential. Sometimes combined editors and compilers (e.g. TeXShop) can be really productive. Make sure you learn the use of synchronized navigation then.

A vector graphic is one where strokes remain strokes even at the highest resolutions: e.g. the Figure **??** or the Logo on the Titelblatt (notice: you can click from here to there). Many tools generate vector-graphics for plots from any data-set. E.g. Plotly (with the use of the Browser-Print), MatPlotLib or even OpenOffice, LibreOffice or MS-Excel.

1.2 Literature References

1.3 Pictures

Task 1: Basic Probabilities and Visualizations (1)

$\xi_1 = 3$

$\xi_2 = 0.61$

A geometric distribution counting the number of Bernoulli trials with p $ \xi_{2} $ until it succeeds.

```
#import libraries
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import geom
```

Step 1:

Define Model

Define $X \sim \text{Geometric}(p)$

and state the Probability Mass Function (PMF): $P(X=k)=(1-\xi_2)^{k-1}\xi_2$.

```
# Define parameters based on xi_2
prob_success=0.61
prob_faliure=1-prob_success
# Initialize the Geometric distribution object (k >= 1)
dist = geom(prob_success)
```

Step 2:

Compute $E[X]$, using the formula $E[X]=1/p$ for $k \geq 1$ (Unit: 2.1, P. 48)

$$E[X]=\frac{1}{0.61} \approx 1.6393$$

```
E_X = 1 / prob_success
print(f"E[X]: {E_X:.4f}")
```

```
E[X]: 1.6393
```

Step 3:

Compute the median M: find the smallest integer k such that the CDF satisfies $P(X \leq k) \geq 0.5$. Use $P(X \leq k)=1-(1-p)^k$.

$ *_{(Unit 2: 2.1, P. 50)}$

We solve for the smallest integer k such that:

$$(1-0.61)^k \leq 0.5$$

$$(0.39)^k \leq 0.5$$

$$k \geq \frac{\ln(0.5)}{\ln(0.39)} \approx 0.736$$

The smallest integer satisfying this is $k=1$.

```
# The median is found using the inverse CDF (Percent Point Function,
ppf)
M = dist.ppf(0.5)
print(f"Median M = {M:.0f}")
```

```
Median M = 1
```

Step 4:

Determine the upper limit K by finding the smallest integer k for which $P(X=k)<0.005$. Explicitly cite the proof steps.

```
threshold = 0.005
k_values = []
# Iterate k starting from 1
for k in range(1, 15):
    prob = dist.pmf(k)
    print(f"P(X={k}) = {prob:.6f}, P(X=k) = {(prob_faliure**(k-
1))*(prob_success)}  Condition P(X=k) < 0.005$? {prob < threshold}")
    if prob < threshold:
        # The range ends at k-1, since k itself is below the
threshold.
        break
    k_values.append(k)

probabilities = dist.pmf(k_values)
K_max = max(k_values)
print(f"The visualization range is k = 1 to {K_max}")
```

```
P(X=1) = 0.610000, P(X=k) = 0.61 Condition P(X=k) < 0.005$? False
P(X=2) = 0.237900, P(X=k) = 0.2379    Condition P(X=k) < 0.005$? False
P(X=3) = 0.092781, P(X=k) = 0.092781  Condition P(X=k) < 0.005$? False
P(X=4) = 0.036185, P(X=k) = 0.03618459Condition P(X=k) < 0.005$? False
P(X=5) = 0.014112, P(X=k) = 0.014111990100000003 Condition P(X=k) <
0.005$? False
P(X=6) = 0.005504, P(X=k) = 0.005503676139000001 Condition P(X=k) <
0.005$? False
P(X=7) = 0.002146, P(X=k) = 0.0021464336942100004     Condition P(X=k)
< 0.005$? True
The visualization range is k = 1 to 6
```

Step 5:

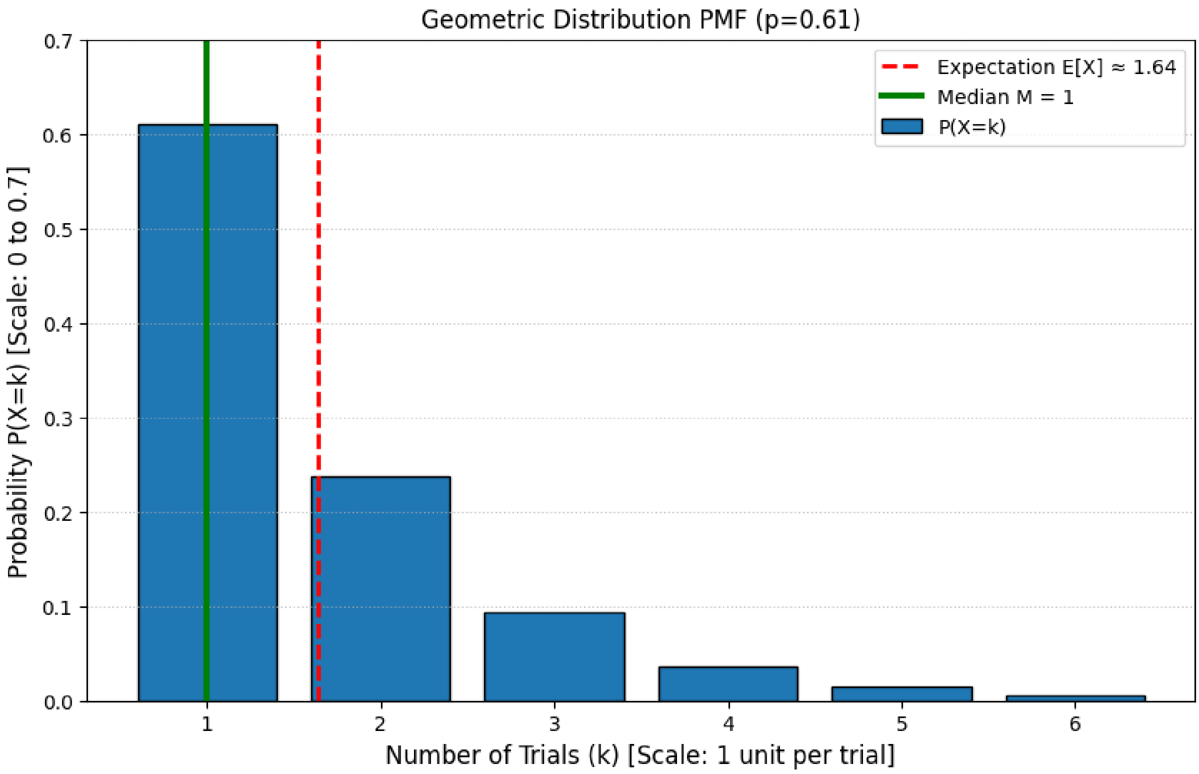
Generate the graphic (bar plot/discrete histogram) for $k=1$ up to K. Include the scale. Present the calculated $E[X]$ and M within the graphic.

We generate a bar plot to visualize the discrete probability mass function (PMF)

$ *_{(Unit 5: 5.5, P. 139)}$

The graphic must include the scale of the axes and clearly display the calculated Expectation and Median

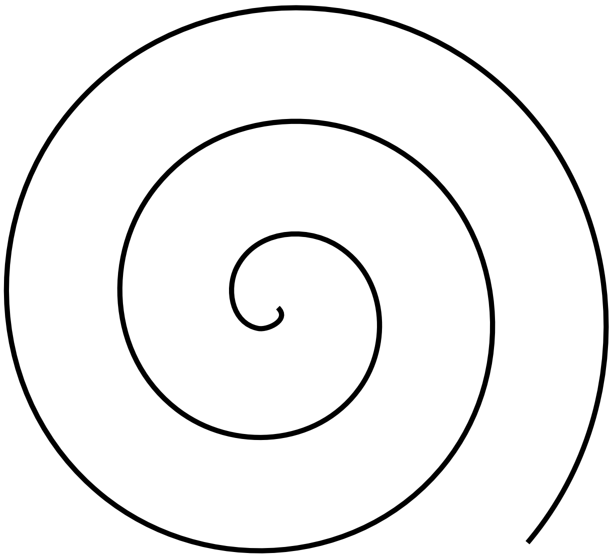
```
plt.figure(figsize=(10, 6))

# Plotting the PMF using a bar plot
plt.bar(k_values, probabilities, color='#1f77b4', edgecolor='black',
label='P(X=k)')

# Mark Expectation
plt.axvline(E_X, color='red', linestyle='--', linewidth=2,
            label=f'Expectation E[X] ≈ {E_X:.2f}')

# Mark Median
plt.axvline(M, color='green', linestyle='-', linewidth=3,
            label=f'Median M = {M:.0f}')

# Formatting and Scale (required by assignment)
plt.title(f'Geometric Distribution PMF (p={prob_success})')
plt.xlabel('Number of Trials (k) [Scale: 1 unit per trial]',
fontsize=12)
plt.ylabel('Probability P(X=k) [Scale: 0 to 0.7]', fontsize=12)
plt.xticks(k_values)
plt.ylim(0, 0.7) # Explicitly setting Y scale for clarity
plt.grid(axis='y', linestyle=':', alpha=0.7)
plt.legend()
plt.show()
```



Justification of Tool Usage:

We utilize Python's standard statistical (scipy) and plotting (matplotlib) libraries.

These tools are widely accepted and trusted for producing accurate numerical results and visualizations in scientific computation.



1.5 Tables

“ Industrial era ”	“Jobs ”	“ Wanted: Upgrade”’
Parts exchanger	Fitter	mecatronics special-ist
eShop	reseller	“ Client-suggester”
“ Coding-guru”’	Softwaredesign	Whole-life designer
JA! Gut & Günstig	brand-names	“ Life-Style Feeling”
Internetbanking	Bank clerk	Customer adviser
Robots	Specialist	Machine supervisor
Bush	Gardener	Nature-sculptor
Painting	Painter	Interior Design

1.6 Listes

- one
- twoi
- threei

1. first
2. second
3. third

1.7 Formulæ

A formula can be written inline, e.g. as $\frac{d}{dx}\mathrm{arctg}(x) = \frac{1}{1+x^2}$ or, in centered math:

$$\frac{d}{dx}\mathrm{arctg}(x) = \frac{1}{1+x^2} \tag{1.1}$$

Notice that formulæ that are centered start bigger (technically, they start in `\displaystyle`) than they start inline (technically, they start in `\textstyle` all subsequents reductions, e.g. an exponent, goes to

`\scriptstyle` then `\scriptscriptstyle`). Indeed a best effort is made so that inline formulæ do not change the line height which would bother the eye of a reader.

Formulæ can be given a number and a label. Numbering happens automatically with `\begin{equation}` and `\end{equation}` and can be avoided if enclosing the formula between `\[` and `\]`. If using the `\label` macro inside, you can refer automatically to this equation using `\ref{label}`. E.g. Thanks to equation 1.1 one dare say that:

$$\int_0^t \frac{1}{1+x^2} dx = \arctan(t) \tag{1.2}$$

1.8 Tools and Code

Many users of this template will want to include some code.

The simplest way to do so is to use the `\verb` macro which is followed by a sign, then some code, then the sign again to close. This is the inline version which works as in:

which yields:

The multiline version of this is called `\begin{verbatim}` and finishes with `\end{verbatim}`.

1.9 Citation examples

Monography (*Advanced Statistics DLMDSAS01*, 2025, p. 22)

1.10 Task 1

Bibliography

Advanced Statistics DLMDSAS01. (2025). IU Internationale Hochschule GmbH, IU International University of Applied Sciences.